

COVER LETTER

Dear Autodesk Tandem Hiring Team,

Omer Tariq Mughal referred me to this role. I want to open with the gap before I make the case: I don't have Go production experience. What I do have is a combination I suspect you haven't seen many times in this application pool, and I'd rather make the case for it directly.

I've spent ten years building systems, and two of those were spent building what I'd describe as the closest existing analogue to what Autodesk Tandem does: a real-time digital representation of a physical system.

At TELUS Digital, I co-led the modernization of a managed pressure drilling (MPD) platform for oil rigs. The system connected PLCs, communications buses, and physical sensors to a real-time UI rendering 10,000+ data points with subsecond tolerance in SVG. Command and status communications flowed through an event-driven architecture with fault-tolerant message brokers. Interpolation algorithms computed precise fluid pressure from sensor readings in real time. Validation schemas enforced data integrity across every boundary between physical reality and its digital representation. The UI had to be trusted as a faithful mirror of a physical system where errors have consequences.

That is, structurally, what a digital twin is.

What makes me different from a backend engineer who has learned React is spatial intuition. Before software, I studied Environmental Design at OCAD and ran an interior design practice. When I look at a digital twin of a building, I'm reading it the same way I read a floor plan — understanding the logic behind the representation, not just the data. I co-developed the design system for the Beyond MPD platform in Figma and built it in code: no handoff layer, no approximation. Component APIs, token architecture, and interaction patterns carried directly to production.

On Go: I'm confident in my ability to ramp. The patterns I know well — event-driven architecture, validation schemas, typed data contracts, service-to-service communication — are language-agnostic in the ways that matter. Go's concurrency model and static typing are meaningful but learnable for an engineer at this level. I'd rather be honest about the ramp than claim experience I don't have.

I appreciate Omer's introduction and look forward to the conversation.

Tony Ko